A flexible and polynomial framework for integer arithmetic in CKKS

Lorenzo Rovida

Politecnico di Torino, Turin, Italy
lorenzo.rovida@polito.it

Abstract. A new paradigm, called *discrete*-CKKS, proposes to restrict the plaintext space of the CKKS scheme from $\mathbb{C}$ to a discrete subset of it (e.g., $\{0, 1\}$). While losing approximate computations, this allows one to express an arithmetic similar to the one available in exact schemes, but with significantly larger parallelism and flexibility due to SIMD computations and the complex arithmetic, that internally is still available. A significant example is the recent work by Boneh and Kim (Crypto '25), where they present a method to operate on extremely large encrypted integers.
In this work, we build a simple computational device that handles integers, decomposed as binary vectors, by evaluating standard mod 2 arithmetic operations using polynomials only. Since we do not resort to the modular reductions based on the functional bootstrapping proposed by Kim and Noh (CIC '25), this makes the parametrization for our framework more flexible and the same as the one used in standard CKKS usages, e.g., leveled with roughly 13 levels before bootstrapping. This means that one can use CKKS in $\mathbb{R}$ and then switch to $\mathbb{Z}$ with the same set of parameters – we will refer to this as *domain-switching*. Experiments show that our solution has lower latency on all operations (i.e., additions, multiplications, comparisons, logical shifts) by roughly three times, with respect to the current state of the art, although we have smaller throughput due to how data is represented.

1 Introduction

Since its introduction by Gentry [23], Fully homomorphic encryption (FHE) has emerged as a very promising technique for computations over encrypted data. Although the first proposals were far from being efficient, today we have a wide range of schemes that can be used to execute programs over ciphertexts according to a given use case; these mainly differ by their plaintext spaces (or domains). All of them share the same core construction based on (Ring-)LWE [38,33], although the way data is encoded and handled dramatically changes the available arithmetic.

The traditional approaches. Since most of lattice-based cryptosystems have natural homomorphic additivity, one of the main historical challenges has been

on how to efficiently evaluate multiplications. Traditionally, there are two different approaches. The first, originally presented in the BGV/BFV schemes [9,7,21], works over the $\mathbb{Z}_p$ plaintext space: this has been one for first practical approaches to FHE. Usually p has some convenient shape to enable SIMD operations [40]. Multiplications are performed as tensor products (or polynomial multiplications, if working with structured lattices) and they are followed by a so-called *relinearization* phase where the size of the resulting ciphertext is reduced back to the original one.

As a second approach, there is a family of schemes whose blueprint was firstly proposed by Ducas and Micciancio [20]. In their proposal, called FHEW, a ciphertext encrypts either 0 or 1. Their approach to FHE is to be able to evaluate a complete Boolean operator followed by a bootstrapping operation (via GSW [24]) that resets the amount of accumulated noise. In their proposal, a NAND operator between two ciphertexts is evaluated by first adding them, and then by applying the map $\{0, 1\} \to 0$ and $2 \to 1$ during the bootstrapping operation. Later on, with TFHE [16,17,18] and its improvements, this has been generalized to small integers and arbitrary look up tables that can be evaluated during the bootstrapping.

Lastly, a BGV-type of scheme, called CKKS [15], allows operations over (approximate) complex numbers in $\mathbb{C}$. This approach has a particular type of encoding based on the inverse discrete Fourier transform (DFT) that enables to work over fixed-point complex numbers that are scaled and rounded to larger integer RLWE polynomials. The DFT encoding ensures SIMD computations over $N/2$ elements, where N is a power of two size of the underlying RLWE ring.

(Some of) the new fresh approaches. In the last years, new approaches have emerged to improve performance of traditional FHE schemes, for example [12,19,13,22,8,27,5,37]. We discuss a couple in the following. Generalizing the blueprint in [12], Geelen and Vercauteren [22] proposed an improved version of BFV by choosing as a plaintext modulus a space of form $\mathbb{Z}[X]/(\Phi_m(x), t(x))$, with $\Phi_m(x)$ being the m-th cyclotomic polynomial and $t(x)$ an arbitrary polynomial. They show that bootstrapping a ciphertext containing 8192 elements over the Fermat prime $\Phi_2(2^{16}) = 2^{16} + 1$ field only takes 2 seconds.

Additionally, a very recent work proposed by Peikert, Zarchy and Zyskind [37], deals with large integers arithmetic using *decomposed*-BFV, where the plaintext modulus can be any integer moduli, including powers-of-two and primes. Multiplications and additions have very low latency (few milliseconds), making it very competitive in terms of latency with respect to other exact-FHE approaches.

1.1 Related works

Our proposal lies in the set of works that enable integer arithmetic over CKKS. We will use $\mathbb{C}$-CKKS (i.e., the "standard" definition of CKKS) and $\mathbb{Z}$-CKKS (i.e., CKKS for integer computation). Even though, depending on the application, one

could resort to more efficient constructions based on exact-FHE, we still believe it is worth to investigate discrete-CKKS as it offers a suite of functionalities that are not available in exact-FHE, more details follow later. Currently, the latest solutions are due to [27,28,5,10].

- [27]: by combining the CKKS modular reduction proposed in [29] with a functional bits bootstrapping framework á la [3], they show how to construct an integer computational device by representing a large integer decomposed in a vector of small chunks. The main bottleneck in their construction is the number of (functional) bootstrapping required to run all the different operations, which is roughly bounded as $O(k \log(k))$, to work with word size 2^k.
- [5]: this approach can be seen as similar to the previous one in terms of ideas (i.e., representing a large number as a set of smaller chunks), but the execution is fairly different. In particular, they use a nested Residue number system (RNS) framework to represent a large number modulo some power-of-radix using the Chinese remainder theorem (CRT). While being more flexible in terms of moduli, this method has smaller latency and throughput than [27] on power-of-two moduli and also has no native way to support operations such as comparisons, logical shifts or LUT evaluations [2].
- [28]: this work is an improvement over [27], done by using two main improvements: (i) the new CinS encoding [25, Section 3] is used to improve how multiplications are performed, in particular they are performed as a convolution-style multiplication and (ii) they rely on lazy modular reduction similarly to [5]. Concretely, they reduce the number of modular reductions from $O(k \log(k))$ down to $O(\log(k))$.
- [10]: in this concurrent paper the authors improve the previous approaches by reducing the number of required bootstrappings to perform computations, achieving similar performance as [28] but enabling the usage of arbitrary moduli and other logical operations.

While achieving impressive performance, we believe that all these works imply a very complex construction which are poorly "flexible", in the sense that if one instantiates discrete-CKKS for large integers, than they will likely lose all the other functionalities given by standard CKKS (i.e., approximate computations over $\mathbb{C}$, smooth functions approximations, etc.). This is a limit set by the usage of modular reduction [29], which requires a bootstrapping operation to perform a modular reduction. Therefore, one typically defines a CKKS parametrization that allows for 1-2 operations before bootstrap. Increasing the number of arbitrary operations is not always possible without increasing the ring size and additionally would imply a loss in overall performance.

1.2 Our contribution

In this paper, we propose a framework that enables integer computations in CKKS with word sizes that are powers of two. We deviate from the standard approaches [5,27,28,10] based on [29] and we propose a solution that is

fully polynomial-based. Differently from similar works that typically instantiate 1-2 levels before bootstrapping, our framework does not require an ad-hoc parametrization and can thus natively enable what we call a *domain-switching*. In practice, we can use CKKS over $\mathbb{R}$ (with 12-13 levels of arbitrary computations), and then convert the encrypted values into integers, and continue computations in $\mathbb{Z}_{2^k}$. Afterwards, one could switch again to $\mathbb{R}$, and so on. To the best of our knowledge, this is the only approach that allows one to achieve practical integers arithmetic without requiring modular reduction based on functional bootstrapping [29].

Our code is publicly available and usable with high-level APIs, written using the OpenFHE implementation of CKKS. We also provide, for readability, the same code in Python using clear vectors based on addition, multiplication and rotation operations.

Simple additions, comparisons and trivial shifts. To reduce the multiplicative depth of additions, we use the Kogge-Stone adder (KSA) [30] as it has logarithmic depth; e.g., for 256-bits addition, this requires concretely $2 + \log_2(256) = 10$ multiplicative levels. Additionally, since we can evaluate a comparison between two values a, b as

$$x \leq y \text{ if and only if } \text{carry}(x - y) = 1, \tag{1}$$

the complexity of the comparison is the same as the one of the addition.

As we will explain later, every binary number is followed by a lot of zeros to improve multiplication efficiency. Hence, a shift can be simply implemented as an automorphism (i.e., a rotation) and has negligible complexity compared to the other operations.

Binary decomposition using polynomials. Concretely, we show how to efficiently convert approximate integer values from $[0, 255] \subset \mathbb{R}$ (with a possible small error) represented in $\mathbb{R}$ format to binary format, using polynomial approximations of some Fourier series. The main idea is that we can find the k-th bit of the decomposition of some integer a by using a function that is what we call *binary k-periodic*, more details are given later. For instance, we can compute the LSB bit $i = 1$ of a number k by using any function that is 2-periodic on $\mathbb{Z}$, taking value 0 on even integers and 1 on odd integers. We provide an intuition of this observation in Figure 1. Remark that computations in CKKS are SIMD, meaning that we can evaluate a different polynomial for each slot, so this operation costs the same as the evaluation of a single polynomial. In practice, these polynomial approximations require 9 levels to be evaluated under Paterson-Stockmeyer [11] with a sufficient level of accuracy. Notice that we significantly improve [19, Algorithm 3] whose decomposition requires to loop over all the bits, and becomes unpractical for large numbers.

A divide-et-impera approach to multiplications. By taking advantage of this practical transformation, we can use, as the core of our general 2^k multiplier,

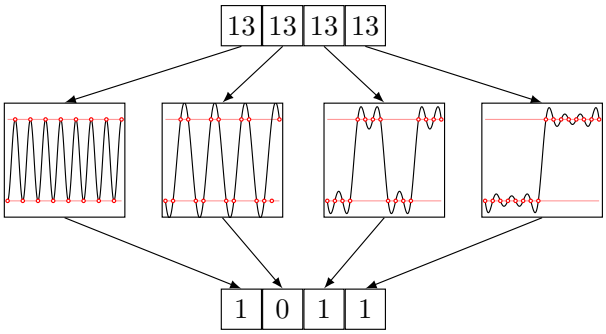


Fig. 1: Decomposing into bits an integer value $x \in [0, 2^k]$ using k polynomial approximations of Fourier series applied to k repetitions of x, with $x = 13, k = 4$

a 4-bits multiplier. Concretely, the actual 4-bits multiplication is performed in $\mathbb{R}$-CKKS and the resulting 8-bits number are transformed to $\mathbb{Z}$-CKKS via polynomials.

In general, for 2^k arithmetic, we use a standard divide-et-impera approach. The two operands $\boldsymbol{a}, \boldsymbol{b} \in \{0, 1\}^k$ are split into high bits $\boldsymbol{a}_h, \boldsymbol{b}_h$ and low bits $\boldsymbol{a}_\ell, \boldsymbol{b}_\ell$; this decomposition reduces the multiplication to four smaller multiplications between the corresponding segments.

This process repeats, halving the size of operands at each step, until it reaches the 4-bit multiplier base case. Then, the partial results are recursively added using a combination of carry-save adders (CSAs) and a KSA to get the final product. We present, in Figure 2, an example of 4-bits multiplication using a 2-bits multiplier as a base case, with $\boldsymbol{a} = [1, 0, 1, 1] \mapsto 13$ and $\boldsymbol{b} = [1, 1, 0, 1] \mapsto 11$.

Notice that we use the least significant bit (LSB) convention, where the leftmost bit represents 2^0, the next bit represents 2^1, and so on. By taking advantage of SIMD computations, the four multiplications can be parallelized, at the cost of requiring $k^2/2$ slots for a 2^k-bit multiplication. In terms of bootstrapping, the complexity of this procedure for 2^k moduli is given by $O(\log(k))$ (the same as [28,10]) as it implies the evaluation of $\log(k)$ additions, and each addition's complexity is $O(1)$.

In praise of domain-switching. An important feature naturally enabled by our framework is what we will call *domain-switching*. In practice, we can handle different data types within the same scheme, conveniently switching between them when needed. We observe that this is not something that is natively possible[1] in previous similar works [5,27,28,10], as their frameworks require a specific parametrization due to the fact that the bootstrapping is *functional*, so it must

[1] Technically, it is possible, but at the cost of adjusting the underlying procedures, increasing the magnitude of parameters and the latency of the operations

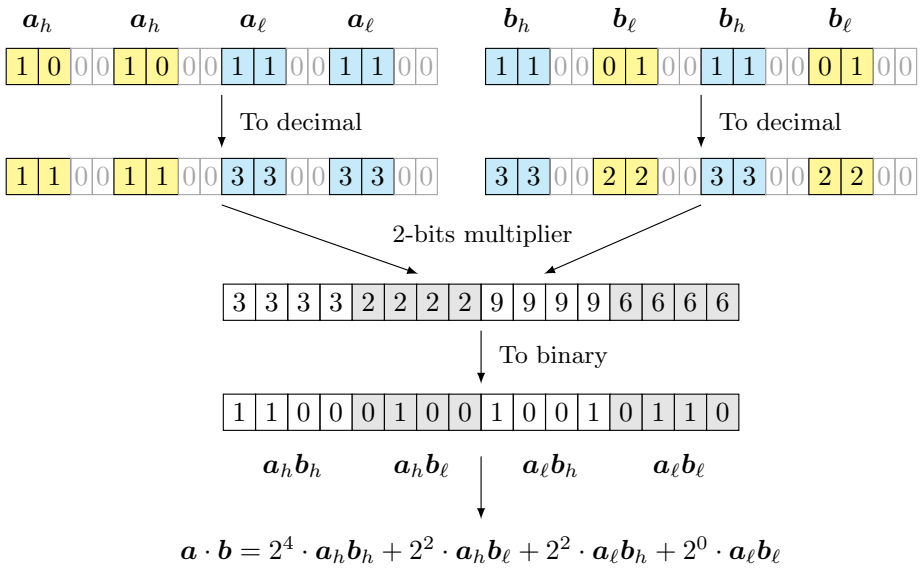


Fig. 2: Reducing $a \cdot b$ to four (shifted) additions. The high bits and low bits are split and repeated for maximum parallelism. The final result is obtained by $(2^4 \cdot 3 + 2^2 \cdot 2 + 2^2 \cdot 9 + 2^0 \cdot 6 = 143 = 11 \cdot 13)$

be evaluated to perform the modular reduction [29]. In our circuit, we also employ a type of "functional" bootstrapping that is only used to reduce the amount of noise in the (binary) ciphertext [3], but this is not a necessary requirement. Technically, one could use the standard CKKS bootstrapping at the cost of including some manual cleaning operations. This means that it could be possible for our setup to evaluate for instance a complete 256-bits multiplication at the cost of $O(1)$ bootstrapping, but by increasing the modulus size (and, in turn, the ring size to $N = 2^{17}$).

We believe the set of possible applications for domain-switching to be pretty large: we briefly review some examples in the following. Typically FHE-based databases employ exact schemes, e.g., [4,43], as they support standard SQL-like operations, differently from approximate schemes. However, this usually limits the types of data that can be stored and homomorphically handled within the database. With our framework, one could store real values in the DB (under $\mathbb{R}$-CKKS) and perform queries based on $\mathbb{Z}$-CKKS, enabling efficient databases containing real-valued values. Additionally, this enables the usage of CKKS-based sorting algorithms [35,39], that are typically orders of magnitude faster than the BFV/TFHE-based ones, to enable practical ORDER BY operations. Also typical Private information retrieval (PIR) constructions [36,31,32] rely on exact FHE (such as BFV or GSW) to perform homomorphic operations. Similarly to the previous case, current solutions allow to retrieve exact results, given the nature of homomorphic operations. By performing queries under $\mathbb{Z}$-

CKKS, one could easily retrieve approximate results whenever needed (e.g., data for recommender systems, features to be used in machine learning models, and so on) as the $\mathbb{R}$-CKKS format is compatible with $\mathbb{Z}$-CKKS.

High-precision smooth functions evaluation in $\mathbb{Z}$. Our framework enables the evaluation of practical high-precision function evaluation in exact FHE in a hybrid way. One can evaluate a first (relatively rough) approximation in $\mathbb{R}$ and refine the result with e.g., Newton-Raphson iterations in $\mathbb{Z}$.

Example 1. Consider the square root function over integers $\sqrt{x}$. In exact schemes, one typically has to use the Newton-Raphson algorithm or similar methods, which can require many iterations over large intervals (e.g., 128-bit integers). In our approach, one can: (i) take advantage of the identity $\sqrt{x} = 2^k\sqrt{x/4^k}$ and approximate $\sqrt{x/4^k}$ using Chebyshev polynomials over some interval; (ii) convert the result to a $\mathbb{Z}$-CKKS ciphertext and perform a left shift by k bits in $\mathbb{Z}$; and (iii) run a few Newton-Raphson iterations to refine the final result.

2 Preliminaries

We use bold lowercase to indicate vectors (or sequence of polynomial coefficients) as $\boldsymbol{a} := (a_0, \dots a_{n-1})$. We consider the subvector from index i to j (excluded) as $\boldsymbol{a}_{i:j}$. Let $\text{bits}(a) : \mathbb{Z}_k \to \{0,1\}^k$ be the bit decomposition function and $\text{dec}(\boldsymbol{x}) : \{0,1\}^k \to \mathbb{Z}$ be its inverse (binary encoding) for some integers $a, k > 0$.

Let $N > 1$ be some power of two and $Q > 1$ an integer modulus. Let $\mathcal{R} := \mathbb{Z}[X]/(X^N+1)$ be the quotient ring of polynomials modulo the $2N$-th cyclotomic polynomial $\Phi_{2n}(x) = X^N + 1$. Also define $\mathcal{R}_Q := \mathcal{R}/Q\mathcal{R}$. These are the required ingredients to build structured q-ary lattices required to use the RLWE [33] trapdoor. In particular, the CKKS scheme [15] is based on a particular type of encoding, called *slots encoding*, in which a discrete Fourier transform (DFT) is used to enable SIMD operations. Let $\text{DFT} : \mathbb{R}[X]/(X^N + 1) \to \mathbb{C}^{N/2}$ be a DFT defined by evaluating a polynomial $p(X)$ in $\left(\xi^{5^i}\right)_{0 \le i < N/2}$ with ξ being a complex primitive $2N$-root of unity and $\text{iDFT} : \mathbb{C}^{N/2} \to \mathbb{R}[X]/(X^N + 1)$ be its inverse.

2.1 CKKS in a nutshell

Key generation in the CKKS scheme follows from RLWE, so we set

$$pk := (\boldsymbol{a} \cdot \boldsymbol{s} + \boldsymbol{e}, \boldsymbol{a}) \in (\mathbb{Z}_Q[X]/(X^N + 1))^2, \quad sk := \boldsymbol{s} \in \mathbb{Z}_Q[X]/(X^N + 1),$$

for uniformly random $\boldsymbol{a} \in \mathcal{R}_Q$ and small $\boldsymbol{s}, \boldsymbol{e} \in \mathcal{R}_Q$. Encryption is performed as in standard RLWE schemes, although encoding in CKKS follows these procedures:

1. Given some plaintext polynomial $\boldsymbol{m}(X) \in \mathbb{C}[X]/(X^{N/2} + 1)$, evaluate an iDFT as $\text{iDFT}(\boldsymbol{m}(X))$ to put $\boldsymbol{m}(X)$ into the so-called *slots-encoding*.

2. The obtained polynomial $\mathrm{iDFT}(m(X))$ is scaled by a large factor $\Delta > 0$, this control the precision of the encoding.
3. Since the polynomial lives in $\mathbb{R}[X]/(X^N+1)$, apply some rounding operation ([34, Section 2.4.2]) to make it compatible with RLWE encryption.

We therefore obtain the following functions:

$$\mathrm{Encode}(\boldsymbol{m}(X)) := \lfloor \Delta \cdot \mathrm{iDFT}(\boldsymbol{m}(X)) \rceil, \quad \mathrm{Decode}(\boldsymbol{e}(X)) := \frac{1}{\Delta} \cdot \mathrm{DFT}\left(\boldsymbol{e}(X)\right)$$

This type of (approximate) encoding enables SIMD operations, since point wise addition or multiplication of plaintext slots corresponds to polynomial addition (or multiplication, respectively) in the iDFT domain via the homomorphisms:

$$\mathrm{DFT}(\mathrm{iDFT}(a) + \mathrm{iDFT}(b)) = a + b \quad \text{and} \quad \mathrm{DFT}(\mathrm{iDFT}(a) \cdot \mathrm{iDFT}(b)) = a \odot b.$$

We refer to Appendix A.1 for more information about CKKS and its (functional) bootstrapping operation.

Discrete-CKKS. A recent line of works, called *discrete*-CKKS [19], propose to restrict the plaintext of CKKS from $\mathbb{C}$ to, e.g., $\{0,1\} \subset \mathbb{C}$. This gives several advantages over exact schemes such as TFHE/BFV, as it allows one to compute over small integers but with a large throughput and to evaluate real valued polynomials over them. We now provide two definitions that will make further discussions lighter

Definition 1 ($\mathbb{R}$-CKKS). *The* $\mathbb{R}$-CKKS *scheme is a restriction of the CKKS scheme in which plaintexts encode vectors in* $\mathbb{R}^{N/2}$.

Definition 2 ($\mathbb{Z}$-CKKS). *The* $\mathbb{Z}$-CKKS *scheme is a restriction of the CKKS scheme in which plaintexts encode – by means of approximate bits* $b_i + e_i$ *for* $b_i \in \{0,1\}$ *and small* $e_i \in \mathbb{R}$ *– vectors in* $(\mathbb{Z}_{2^b})^k$ *for some integer* $b > 0$ *and* $k < N/2$.

2.2 Conversions between $\mathbb{Z}$-CKKS and $\mathbb{R}$-CKKS

Given a $\mathbb{Z}$-CKKS ciphertext, it is easy to convert it into a $\mathbb{R}$-CKKS ciphertext, given that the $\Delta > 0$ parameter is sufficiently large to accommodate the converted value.

Lemma 1 ($\mathbb{Z}$-CKKS to $\mathbb{R}$-CKKS conversion). *Given a* $\mathbb{Z}$-CKKS *ciphertext encoding some vector* $\boldsymbol{x} \in \mathbb{Z}_k$*, with each* $x_i \in \{0,1\}$*, there exists an algorithm that converts it into a* $\mathbb{R}$-CKKS *ciphertext with* $\log(k)$ *additions and rotations.*

The proof can be found in Appendix D.1, simply observe that by multiplying the binary $\mathbb{Z}$-CKKS ciphertext with a vector shaped as $(2^0, 2^1, \dots)$, the weighted sum will exactly contain the real value. On the other hand, the inverse conversion is less immediate as it involves a binary decomposition. A possible algorithm would be to compute a nonlinear function (such as the parity function) for

k times to convert a value in $\mathbb{Z}_{2^k}$ into a k-bits one (e.g., [19, Algorithm 3]). However, this becomes unpractical as it requires the evaluation of $O(k)$ nonlinear functions which can not be parallelized. Our goal is to take advantage of SIMD parallelization to perform this conversion in a single step.

We thus propose a novel approach to perform such decomposition by means of discrete Fourier series. In particular, we observe that the binary decomposition of some value in (some restriction of) $\mathbb{Z}$ can be evaluated by extracting each bit independently, using periodic functions over 0 and 1. Let us start from the following.

Lemma 2. *The k-th bit of some value $x \in \mathbb{Z}_{2^k}$ is equal to $\lfloor x/2^k \rfloor \bmod 2$.*

Refer to Appendix D.2 for the proof. Intuitively, observe that in the binary representation, each bit k corresponds to the coefficient of 2^k. When an integer is increased by one, the LSB changes at every step, while more significant bits change only when the count crosses a multiple of 2^k. As a result, bit k remains constant for 2^k consecutive values, then flips and stays in the new state for another 2^k values. This pattern repeats cyclically, producing a periodic sequence with period 2^{k+1}. To describe such behavior, we introduce the concept of *binary k-periodic function*.

Definition 3 (Binary k-periodic function). *A function $f(x) : \mathbb{R} \to \mathbb{R}$ is said to be* binary k-periodic *if, for all $x \in \mathbb{Z}$, it holds that*

$$f(x) = \lfloor x/2^k \rfloor \bmod 2.$$

In particular, it has period 2^{k+1}, where the first 2^k samples are $(0, 0, \dots)$ while the second 2^k are $(1, 1, \dots)$.

Of course $\lfloor x/k \rfloor \bmod 2$ itself is binary k-periodic, but it is hard to approximate it through polynomials as it contains many steps and it is highly non linear. Since Lemma 2 is defined over integer inputs, we easily derive the following.

Corollary 1. *The k-th bit of some value $x \in \mathbb{Z}^+$ is equal to $f(x)$, where f is a binary k-periodic function.*

Square waves and Fourier series. To tackle this issue, we first notice that the (sub)set of points required to satisfy the condition in Definition 3 – namely the integers – are equidistant points. This naturally leads to discrete Fourier transforms. In general, the DFT takes a finite set of equidistant samples of a function and represents them as a finite linear combination of complex exponentials. If the samples arise from a function defined on an interval $[0, 2P]$, the corresponding frequencies are integer multiples of π/P. For real-valued data, this representation can equivalently be written as a *trigonometric polynomial*.

Definition 4 (Trigonometric polynomial). *Let $n \in \mathbb{N}^+$. A $2P$-periodic* trigonometric polynomial *of degree n is a function of the form*

$$p(x) = \frac{a_0}{2} + \sum_{k=1}^{n} \left(a_k \cos\left(\frac{k\pi x}{P}\right) + b_k \sin\left(\frac{k\pi x}{P}\right) \right),$$

where $a_k, b_k \in \mathbb{R}$.

Our idea is to express a binary k-periodic function as a trigonometric polynomial by interpolating its values at the integers in $[0, 2P]$. Since the notion of binary k-periodicity is defined through these integer sample points, the resulting trigonometric polynomial will again be binary k-periodic. The interpolant is therefore smooth and $2P$-periodic, which makes it more suitable to further approximation, e.g., by Chebyshev polynomials.

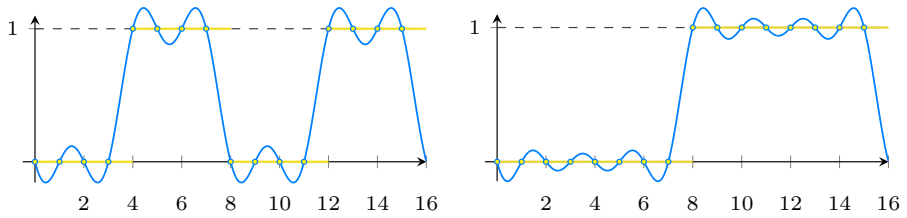
Theorem 1 ([41, Theorem 2.3.1.5], adapted). *For any support points* $(x_i, f(x_i))$*, with* $0 \leq i < N$*,* f *real and* $x_i = 2i\pi/N$ *(equidistant), there exists a unique trigonometric polynomial* $p(x)$ *with*

$$p(x_i) = f(x_i) \quad \textit{for } 0 \leq i < N.$$

Given that Theorem 1 enables to define a unique trigonometric polynomial that interpolates through all the integers in $[0, 2P]$, we can derive, as a consequence, that a trigonometric polynomial constructed from the equidistant integer samples of some binary k-periodic function is again binary k-periodic.

Corollary 2. *There exists some* $f(x)$ *such that the unique trigonometric polynomial* $p(x)$ *interpolating* $f(x)$ *is binary* k*-periodic.*

The proof can be found in Appendix D.4. By combining this result with previous observations, we can use the DFT of $\lfloor x/2^k \rfloor \bmod 2$ – sampled on the integers in $[0, 2^b] \subset \mathbb{Z}$ – to extract the k-th bit of some integer, with $k \leq b$. Although the target function is discontinuous, the DFT produces a smooth trigonometric polynomial. This enables the construction of polynomial approximations for discrete bit extraction via Chebyshev polynomials. In Figure 3 we illustrate, as an example, two trigonometric polynomials, computed via DFT, that can be used to extract the 3rd and the 4th bit of an integer in $\mathbb{Z}_{16}$.



(a) $2^k = 4$, allows one to extract the 3rd bit

(b) $2^k = 8$, allows one to extract the 4th bit

Fig. 3: In yellow the plot of $\lfloor x/2^k \rfloor \bmod 2$, in blue its Fourier expansion derived from a DFT, in the $[0, 16]$ interval. This expansion allows one to extract the $(k+1)$-th bit (with $k = 4 = \log(16)$) from an input $x \in [0, 16]$

What we will later do concretely is to approximate such functions over the $[0, 256]$ interval, to extract the k-th bit of an integer $x \in \mathbb{R} \subset [0, 256]$. Of course

the main limit of such conversion is that converting integer values $x \in [0, 2^k] \subset \mathbb{R}$ requires a polynomial approximation in the interval $[0, 2^k]$, whose size grows exponentially with k. Since we will use, as base case for our multiplications, a 4-bits multiplier based on $\mathbb{R}$-CKKS, we will need to convert the result (which will be an 8-bits value in $[0, 225] \subset \mathbb{Z}$) back to $\mathbb{Z}$-CKKS with such polynomial approximations.

Finally, we can put together all the obtained results to define a conversion between $\mathbb{R}$-CKKS to $\mathbb{Z}$-CKKS.

Lemma 3 ($\mathbb{R}$-CKKS to $\mathbb{Z}$-CKKS conversion). *Given a* $\mathbb{R}$-CKKS *ciphertext encoding some value* $x + \varepsilon$ *with* $x \in \mathbb{Z}_{2^b}$ *and small* $\varepsilon \in \mathbb{R}$*, there exists an algorithm that converts it into a* $\mathbb{Z}$-CKKS *ciphertext by evaluating each* k*-th copy of* x*, with* $0 < k \leq b$*, using a polynomial approximation of a binary* k*-periodic trigonometric polynomial. The precision of the* $\mathbb{Z}$-CKKS *ciphertext depends on* ε *and the accuracy of the polynomial approximation.*

The proof, based on the just presented reasoning, can be found in Appendix D.3.

3 Methodology

In this section we provide some algorithmic definitions of the primitives enabled by our framework, including binary additions ($a+b$), comparisons ($a \leq b$), logical shifts ($a \ll k$) and multiplications ($a \cdot b$).

3.1 Additions

As previously mentioned, additions are performing using the Kogge Stone adder (KSA) [30], described in Algorithm 1. It turns out that this algorithm well fits

Algorithm 1 Kogge Stone adder (KSA)

Require: Two n-bit numbers $\boldsymbol{a} = [a_0, \ldots, a_{n-1}]$, $\boldsymbol{b} = [b_0, \ldots, b_{n-1}]$
Ensure: Sum $\boldsymbol{s} = [s_0 \ldots s_{n-1}]$ and carry-out s_n
Procedure KSA($\boldsymbol{a}, \boldsymbol{b}$)
  *Define the propagate (**p**) and generate (**g**) vectors:*
  $p_i := a_i \oplus b_i$
  $g_i := a_i \wedge b_i$
  Compute prefix carries using Kogge-Stone parallel prefix:
for $d := 0$ **to** $\lceil \log_2 n \rceil - 1$ **do**
    $g_i := g_i \vee (p_i \wedge g_{i-2^d})$
    $p_i := p_i \wedge p_{i-2^d}$
end for
  Post processing:
  $g_n := 0$
  $s_i := p_i \vee g_{i-1}$
return $\boldsymbol{s} := [s_0 \ldots s_{n-1}, s_n]$

the SIMD schemes, as it enables the computation of most of the workload in parallel.

Initial propagate and generate. The first two vectors, $\boldsymbol{p}$ (propagate) and $\boldsymbol{g}$ (generate), are obtained respectively by computing $p_i = a_i \oplus b_i$ and $g_i = a_i \wedge b_i$. In our case, we can directly write $\boldsymbol{p} := \boldsymbol{a} + \boldsymbol{b} \bmod 2$ and $\boldsymbol{g} = \boldsymbol{a} \cdot \boldsymbol{b}$, which can be made to require only one multiplicative level. Refer to Section A.2 for details about the different approaches to evaluate the mod 2 operation in CKKS.

Computing prefix carries. This is the core and main loop of the KSA. Let us analyze the two lines 7-8 of Algorithm 1. Firstly, line 7 assigns, at g_i, the value of

$$g_i \vee (p_i \wedge g_{i-2^d}).$$

The internal AND can be evaluated as $\boldsymbol{p} \cdot \mathrm{Rot}_{2^d}(\boldsymbol{g})$, where $\mathrm{Rot}_i(\cdot)$ is the CKKS rotation by i positions to the right. On the other hand, the external OR would require us to perform (at least) an additional multiplication to reduce mod 2. However, we observe that such mod 2 is not required because the two terms are guaranteed never to be 1 at the same time. The reason is that if $g_i = 1$, it indicates a carry has already been generated from a lower-order bit that includes all the bits covered by $g_{i-2^{d/2}}$. Therefore, g_i and $p_i \wedge g_{i-2^d}$ can never overlap, making addition equivalent to OR for this line. Hence, we can evaluate the line simply as $\boldsymbol{g} := \boldsymbol{g} + (\boldsymbol{p} \cdot \mathrm{Rot}_{2^d}(\boldsymbol{g}))$, consuming one multiplicative depth. Secondly, line 8 assigns, to p_i, the value of

$$p_i \wedge p_{i-2^d}.$$

The latter can easily be implemented as $\boldsymbol{p} := \boldsymbol{p} \cdot \mathrm{Rot}_{2^d}(\boldsymbol{p})$. Since the two lines do not depend on each other, each loop iteration consumes only one multiplicative depth.

Post processing. This is the final step and it is common to all adders of this family (carry look ahead). It involves computation of sum bits using OR operations. Line 11 manually sets the value of $g_n = 0$, and this would require us to mask all the positions but n, at the cost of one multiplication. We avoid this by simply assuming that the bits representation of the resulting value is followed by a 0. This ensures that, when line 12 uses the value of g_n, it will use the 0 we manually injected. Notice that this is compatible with the multiplications logic that will require some zeros after each number, more details will be given later.

Lemma 4 (KSA in CKKS). *There exists an algorithm that evaluates the KSA requiring* $2 + \log(n)$ *multiplicative levels to add two* n*-bits numbers.*

The proof can be found in Appendix D.5, and for the sake of completeness, we also give in Algorithm 3 a version of the KSA specifically written using CKKS primitives.

3.2 Comparisons

As already mentioned in Eq. (1), comparisons can be implemented as a subtraction. Given two n-bits numbers $\boldsymbol{a} = (a_0, \dots a_{n-1})$ and $\boldsymbol{b} = (b_0, \dots, b_{n-1})$, we first reverse the bits of $\boldsymbol{b}$ as $\hat{b}_i = 1 - b_i$, and then we add $\boldsymbol{a} + \hat{\boldsymbol{b}}$ using e.g., KSA. The result of the comparison will be the last carry present in the n-th position. We formalize this operation in Algorithm 4.

3.3 Multiplications

Our goal is to evaluate multiplications by taking advantage of SIMD computations, not to improve throughput, but to improve latency. For this reason we will require more than n slots to represent an n-bits number. We further elaborate below. Our reference construction is a divide-et-impera multiplier as it is (i) highly parallelizable, which is good for SIMD computations and (ii) has logarithmic depth, which is good to reduce the number of bootstrappings. In particular, instead of using Karatsuba divide-et-impera approach – that reduces the number of multiplications to three – we use the standard textbook one as Karatsuba requires to perform more additions, which are the main bottleneck in our algorithm. As a first step, observe that we need to preprocess the inputs $\boldsymbol{a}, \boldsymbol{b} \in \{0,1\}^n$ by defining high bits $(\boldsymbol{a}_h, \boldsymbol{b}_h)$ and low bits $(\boldsymbol{a}_\ell, \boldsymbol{b}_\ell)$ as illustrated in Figure 4. Each operand is followed by zeroes required to accommodate the

Fig. 4: Processing $\boldsymbol{a}, \boldsymbol{b} \in \{0,1\}^n$ before the multiplication.

partial result (remark that a $n/2$-bits multiplication returns n bits, see Figure 2 for a reference), observe indeed that each vector now requires $4n$ slots. We can now compute the actual multiplication as:

$$2^n \cdot \boldsymbol{a}_h \cdot \boldsymbol{b}_h + 2^{n/2} \cdot \boldsymbol{a}_h \cdot \boldsymbol{b}_\ell + 2^{n/2} \cdot \boldsymbol{a}_\ell \cdot \boldsymbol{b}_h + \boldsymbol{a}_\ell \cdot \boldsymbol{b}_\ell. \tag{2}$$

In particular, we use a 4-bit multiplier as the base case and evaluate all larger multiplications recursively. Power of two multiplications (logical shifts) are evaluated as simple rotations.

The base case ($n = 4$). We implement the base case using the approach anticipated in Section 2.2. Given two values $\boldsymbol{a}, \boldsymbol{b} \in \{0,1\}^4$, we convert them to $\mathbb{R}$-CKKS ciphertexts (Lemma 1) and we perform the standard CKKS multiplication. The result will approximately lie in $[0, 225] \subset \mathbb{Z}$, and we will convert it back to $\mathbb{Z}$-CKKS (Lemma 3).

The recursive case ($n > 4$). Following the divide-et-impera approach, we split the multiplication in four smaller multiplications based on the $n/2$-bits multiplier, until we reach the base case $n = 4$. Then, for each call, we require to evaluate four n-bits additions using the KSA. We can reduce the four additions to a single one, after two rounds of carry-save adder (CSA), additional details are given in Appendix B.

Since we want to parallelize every operation, we want to reduce all the base cases into a single ciphertext. To begin, we need to accommodate the four $n/2$-bits operands, plus the same amount of zeroes, we will thus start with the requirement of $4n$ slots. Subsequently, since we will recursively split each operand in high and low bits of size $n/2$, each of the four operands will recursively require four times half their space, for a total of twice the slots required for each recursive call, refer to Figure 5 for a visual intuition.

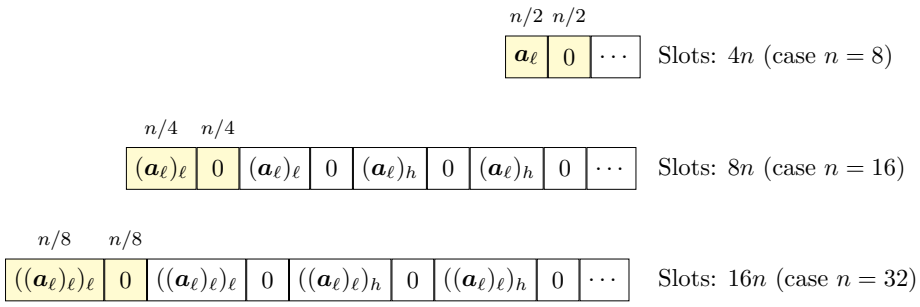


Fig. 5: The number of required slots to allow for parallelization doubles at each recursive call

We provide a pseudo code of the proposed divide-ed-impera multiplication algorithm in Algorithm 2. As for additions, we refer to a pure SIMD definition

Algorithm 2 Recursive algorithm for n-bits multiplications

Procedure mult($\boldsymbol{a}, \boldsymbol{b}, n$)
  if $n = 4$ **then**
    Base case:
    return $\mathbb{R}$-to-$\mathbb{Z}$($\mathbb{Z}$-to-$\mathbb{R}$($\boldsymbol{a}$) $\cdot$ $\mathbb{Z}$-to-$\mathbb{R}$($\boldsymbol{b}$))
  else
    Recursive case:
    $\boldsymbol{a}_\ell := \boldsymbol{a}_{1,n/2}, \quad \boldsymbol{a}_h = \boldsymbol{a}_{n/2,n}$
    $\boldsymbol{b}_\ell := \boldsymbol{b}_{1,n/2}, \quad \boldsymbol{b}_h = \boldsymbol{b}_{n/2,n}$
    return $2^n \cdot \text{mult}(\boldsymbol{a}_h, \boldsymbol{b}_h, n/2) + 2^{n/2} \cdot \text{mult}(\boldsymbol{a}_\ell, \boldsymbol{b}_h, n/2) + 2^{n/2} \cdot \text{mult}(\boldsymbol{a}_h, \boldsymbol{b}_\ell, n/2) + \text{mult}(\boldsymbol{a}_\ell, \boldsymbol{b}_\ell, n/2)$

of the recursive algorithm in Algorithm 5.

Lemma 5. *An n-bits multiplication, with $n = 2^k$ and $k > 2$, requires $n^2/2$ slots to be executed in parallel.*

The proof is given in Appendix D.6. This result implies that the concrete throughput of our approach is severely limited by this constraint.

Corollary 3. *Given s available* CKKS *slots, one can represent up to $2s/n^2$ distinct n-bit integers in* $\mathbb{Z}$-CKKS *format that are compatible with parallel divide-et-impera multiplication.*

On the size of the result of the multiplication. Observe that our multiplication between two n-bits numbers naturally gives an n^2-bits number. This can be desirable in some applications, although it would require an additional operation to split the results into more ciphertexts to allow for further multiplications, as the number of slots decreases when n is increased. On the other hand, one can also ignore the last n bits of the result to obtain the value modulo 2^n and keep performing computations. This is the strategy we followed in our experiments.

3.4 Logical shifts

Given that, to evaluate fast multiplications in parallel, we require each number to be followed by an amount of zeroes (larger than n), we can simply compute a logical shift as a rotation, thus

$$x \ll k \text{ is implemented as } \mathrm{Rot}_{-k}(x). \tag{3}$$

This can change some values that are supposed to be equal to zero between the encoded numbers, although they are implicitly masked for a multiplication. Concerning additions, one could clean the old values with a masking operation that increases by one the multiplicative depth, but this does not increase the complexity which stays to $O(1)$ bootstrappings.

4 Experiments

We implemented our framework using OpenFHE [1,26], and the code is publicly available in GitHub[2]. The repository provides an equivalent implementation in a Python notebook with plaintext values, intended for illustrative purposes. We structured the source code with high-level APIs so that it easy to perform operations in $\mathbb{Z}$-CKKS. All experiments have been executed in single-thread on a Macbook with M4 Max machine with 32 GB of memory and are easily replicable. Although we perform experiments on power-of-two word sizes, our polynomial framework can be adapted to work with special primes, refer to Appendix C.1.

Parametrization. As anticipated in Section 1.2, one of the main points of our proposal is that CKKS is instantiated with a set of parameters which is somewhat generic, i.e., it is not ad-hoc for the integer framework. We present in Table 1 the set of chosen parameters. Observe that the available depth (i.e., $29 - 15$)

[2] github.com/lorenzorovida/flexible-integer-arithmetic-ckks

Table 1: The CKKS parameters used for the experiments with 64-bits, refer to [26] for details about the parameters. BTS stands for bootstrapping. For the cases of 128 (and 256) bits, we increase the depth by one (two, resp.)

$\log(N)$	$\log(\Delta)$	$\log(QP)$	d_{num}	Secret key distribution	Total depth	BTS depth
16	40	1730	3	Sparse encapsulated	30	15

leaves 14 levels of arbitrary computations outside of the bootstrapping, enabling the evaluation of any circuit in $\mathbb{R}$-CKKS prior or after evaluations in $\mathbb{Z}$-CKKS. Next, we provide in Table 2 some details about how each moduli of the chain is used. Remark that we use the StC-first type of bootstrapping, which enables the usage of "cleaning" bootstrapping [3].

Remark 1. We used the same set of parameters for each experiment, although better performance can easily be achieved by setting specific parameters for bits smaller than 256. We report in Appendix C some additional runtimes achieved with specific sets of parameters depending on the number of bits.

Table 2: Usage of moduli in the moduli chain under $\mathbb{Z}$-CKKS and $\mathbb{R}$-CKKS. The $\cos(x)$-like bootstrapping refers to the cleaning bootstrapping introduced in [3], while the $\sin(x)$-like to the standard CKKS bootstrapping [6]. First approx. refers to an initial Chebyshev approximation, followed by double angle iterations to improve the precision

Mode	BTS type	CtS	First approx. $d = 27$	Double angle	Arbitrary	StC
$\mathbb{Z}$-CKKS	$\cos(x)$-like	4	5	3	14	3
$\mathbb{R}$-CKKS	$\sin(x)$-like	4	5	3	14	3

Evaluating integer operations. We now aim to assess our framework with respect to the current state-of-the-art work regarding integer arithmetic in CKKS [27,28,5,10]. We perform different types of experiment, one for each arithmetic operation, and we report the runtime required to compute the circuit and to return the bootstrapped result, ready to be used for other computations. All experiments can be reproduced along with the examples provided in the repository. We avoided a direct comparison with [5] as their RNS work is mostly aimed at arbitrary moduli and does not perform well on power-of-two moduli. Moreover, [5] does not have the same versatility as ours or [28,10] as there is no "native" way to support non-arithmetic operations such as comparisons, logical shifts or LUT evaluations [2]. See Table 3 for results about additions.

Concerning multiplications (Table 4), note that TFHE [42] always works on a single slot, and its performance has been reported from [27], as they run the

Table 3: Evaluating the addition operations $a + b$ in terms of bootstrappings (BTS), runtime and available slots. The runtimes for [28,10] are expected as they did not provide any data or source code

	This work			[28]			[10]		
Bits	Time	Slots	BTS	Time	Slots	BTS	Time	Slots	BTS
16	2.72 sec	256	1	$\approx$ 12.5 sec	8192	2	$\approx$ 13 sec	4096	1
32	3.13 sec	64	1	$\approx$ 12.5 sec	4096	2	$\approx$ 13 sec	2048	1
64	3.44 sec	16	1	$\approx$ 12.5 sec	2048	2	$\approx$ 13 sec	1024	1
128	3.88 sec	4	1	$\approx$ 12.5 sec	1024	2	$\approx$ 13 sec	512	1
256	4.05 sec	1	1	$\approx$ 12.5 sec	512	2	$\approx$ 13 sec	256	1

Table 4: Evaluating the multiplication operations $a \cdot b$ in terms of bootstrappings (BTS) and runtime. Notice that for 32/64-bits operations, our proposal is at least 30% faster than other works based on CKKS

	This work			[42]	[28]			[10]		
Bits	Time	Slots	BTS	Time	Time	Slots	BTS	Time	Slots	BTS
16	10.5 sec	256	2	1.62 sec	25.3 sec	8192	4	39.89 sec	4096	3
32	14.0 sec	64	3	6.42 sec	24.9 sec	4096	4	39.78 sec	2048	3
64	18.9 sec	16	4	25.4 sec	24.9 sec	2048	4	40.30 sec	1024	3
128	21.7 sec	4	5	101 sec	30.9 sec	1024	5	57.41 sec	512	4
256	25.3 sec	1	6	403 sec	31.9 sec	512	5	74.76 sec	256	5

benchmarks with the same machine we used. We considered the result modulo 2^n by discarding the last n bits, although one can easily retrieve the real product modulo 2^{2n} without major changes.

Logical shifts. As already mentioned in Section 3.4, in our setup shifts are simply evaluated as rotations (roughly 0.1 sec). On the other hand, there is no efficient way to perform arbitrary shifts in [28], while in [27] the runtime for a 64-bits shift is around 100 seconds. Lastly, in [10], for some specific shifts it is possible to use rotations, but this is limited to multiples of B (in their experiments $B = 16$).

Evaluating comparison operations. We now aim to assess the performance with respect to the operations of comparison and logical shift. Notice that, as specified in Section 3.2, in our framework comparisons can be implemented with the same complexity as additions. Additionally, logical shifts are implemented simply as CKKS rotations. We compare with [27] as comparisons are more efficient than in [28]. In particular they require $\approx 2u$ bootstrappings, with $u = n/4$ (refer to [27, Table 3]).

Table 5: Evaluating the comparison operations ($a \leq b$) in terms of bootstrappings (BTS) and runtime. Results for [27] are all expected by considering the number of BTS, except for the case of 64-bits that was reported by the authors

	This work			[27]		
Bits	Time	Slots	BTS	Time	Slots	BTS
16	2.72 sec	256	1	$\approx$ 25 sec	16384	8
32	3.13 sec	64	1	$\approx$ 50 sec	16384	16
64	3.44 sec	16	1	102 sec	16384	32
128	3.88 sec	4	1	$\approx$ 198 sec	16384	64
256	4.05 sec	1	1	$\approx$ 395 sec	16384	128

References

1. Al Badawi, A., et al.: OpenFHE: Open-Source Fully Homomorphic Encryption Library. In: Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography. pp. 53–63 (2022)
2. Alexandru, A., Kim, A., Polyakov, Y.: General functional bootstrapping using ckks. In: Advances in Cryptology – CRYPTO 2025. pp. 304–337 (2025)
3. Bae, Y., Cheon, J.H., Kim, J., Stehlé, D.: Bootstrapping bits withckks. In: Advances in Cryptology – EUROCRYPT 2024. pp. 94–123 (2024)
4. Bian, S., Zhang, Z., Pan, H., Mao, R., Zhao, Z., Jin, Y., Guan, Z.: He3db: An efficient and elastic encrypted database via arithmetic-and-logic fully homomorphic encryption. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 2930–2944 (2023)
5. Boneh, D., Kim, J.: Homomorphic encryption forlarge integers fromnested residue number systems. In: Advances in Cryptology – CRYPTO 2025. pp. 338–370 (2025)
6. Bossuat, J.P., Troncoso-Pastoriza, J., Hubaux, J.P.: Bootstrapping forapproximate homomorphic encryption withnegligible failure-probability byusing sparse-secret encapsulation. In: Applied Cryptography and Network Security. pp. 521–541 (2022)
7. Brakerski, Z.: Fully homomorphic encryption without modulus switching from classical gapsvp. In: Advances in Cryptology – CRYPTO 2012. pp. 868–886 (2012)
8. Brakerski, Z., Friedman, O., Golan, D., Gurny, A., Mutzari, D., Sheinfeld, O.: REFHE: Fully homomorphic ALU. Cryptology ePrint Archive, Paper 2025/1449
9. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. In: Proceedings of the 3rd Innovations in Theoretical Computer Science Conference. pp. 309–325 (2012)
10. Cha, G., Park, D., Lee, J.W.: Improved radix-based approximate homomorphic encryption for large integers via lightweight bootstrapped digit carry. Cryptology ePrint Archive, Paper 2025/1740 (2025)
11. Chen, H., Chillotti, I., Song, Y.: Improved bootstrapping for approximate homomorphic encryption. In: Advances in Cryptology – EUROCRYPT 2019. pp. 34–54 (2019)
12. Chen, H., Laine, K., Player, R., Xia, Y.: High-precision arithmetic in homomorphic encryption. In: Cryptographers Track at the RSA Conference. pp. 116–136 (2018)
13. Cheon, J.H., Cho, W., Kim, J., Stehlé, D.: Homomorphic multiple precision multiplication for ckks and reduced modulus consumption. In: Proceedings of the 2023

ACM SIGSAC Conference on Computer and Communications Security. pp. 696–710 (2023)
14. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: Bootstrapping for approximate homomorphic encryption. In: Advances in Cryptology – EUROCRYPT 2018. pp. 360–384 (2018)
15. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic encryption for arithmetic of approximate numbers. In: Advances in Cryptology – ASIACRYPT 2017. pp. 409–437 (2017)
16. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In: Advances in Cryptology – ASIACRYPT 2016. pp. 3–33 (2016)
17. Chillotti, I., Joye, M., Paillier, P.: Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In: Cyber Security Cryptography and Machine Learning. pp. 1–19 (2021)
18. Chillotti, I., Ligier, D., Orfila, J.B., Tap, S.: Improved programmable bootstrapping withlarger precision andefficient arithmetic circuits fortfhe. In: Advances in Cryptology – ASIACRYPT 2021. pp. 670–699 (2021)
19. Drucker, N., Moshkowich, G., Pelleg, T., Shaul, H.: Bleach: Cleaning errors in discrete computations over ckks. J. Cryptol. **37**(1) (11 2023)
20. Ducas, L., Micciancio, D.: Fhew: Bootstrapping homomorphic encryption in less than a second. In: Advances in Cryptology – EUROCRYPT 2015. pp. 617–640 (2015)
21. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Paper 2012/144 (2012)
22. Geelen, R., Vercauteren, F.: Fully homomorphic encryption for cyclotomic prime moduli. In: Advances in Cryptology EUROCRYPT 2025. pp. 366–397 (2025)
23. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing. pp. 169–178 (2009)
24. Gentry, C., Sahai, A., Waters, B.: Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In: Advances in Cryptology – CRYPTO 2013. pp. 75–92 (2013)
25. Ju, J.H., Park, J., Kim, J., Kang, M., Kim, D., Cheon, J.H., Ahn, J.H.: Neujeans: Private neural network inference with joint optimization of convolution and fhe bootstrapping. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. pp. 4361–4375 (2024)
26. Kim, A., Papadimitriou, A., Polyakov, Y.: Approximate homomorphic encryption with reduced approximation error. In: Topics in Cryptology CT-RSA 2022: Cryptographers Track at the RSA Conference 2022. pp. 120–144 (2022)
27. Kim, J.: Efficient homomorphic integer computer from ckks. IACR Transactions on Cryptographic Hardware and Embedded Systems **2025**(4), 873898 (9 2025)
28. Kim, J.: Faster homomorphic integer computer. Cryptology ePrint Archive, Paper 2025/1440 (2025)
29. Kim, J., Noh, T.: Modular reduction in CKKS. IACR Communications in Cryptology **2**(2) (2025)
30. Kogge, P.M., Stone, H.S.: A parallel algorithm for the efficient solution of a general class of recurrence equations. IEEE Trans. on Comp. **C-22**(8), 786–793 (1973)
31. Li, B., Micciancio, D., Raykova, M., Schultz-Wu, M.: Hintless single-server private information retrieval. In: Annual International Cryptology Conference – CRYPTO '24. pp. 183–217 (2024)

32. Luo, M., Liu, F.H., Wang, H.: Faster fhe-based single-server private information retrieval. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. pp. 1405–1419 (2024)
33. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. J. ACM **60**(6) (11 2013)
34. Lyubashevsky, V., Peikert, C., Regev, O.: A toolkit for ring-lwe cryptography. In: Advances in Cryptology – EUROCRYPT 2013. pp. 35–54 (2013)
35. Mazzone, F., Everts, M., Hahn, F., Peter, A.: Efficient ranking, order statistics, and sorting under ckks. In: 34th USENIX Security Symposium (USENIX Security 25). pp. 8541–8558 (2025)
36. Menon, S.J., Wu, D.J.: Spiral: Fast, high-rate single-server pir via fhe composition. In: 2022 IEEE symposium on security and privacy (S&P). pp. 930–947 (2022)
37. Peikert, C., Zarchy, D., Zyskind, G.: High-precision exact FHE made simple, general, and fast. Cryptology ePrint Archive, Paper 2025/2321 (2025)
38. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. J. ACM **56**(6) (9 2009)
39. Rovida, L., Leporati, A., Basile, S.: Lightweight sorting in approximate homomorphic encryption. Cryptology ePrint Archive, Paper 2025/1150 (2025)
40. Smart, N.P., Vercauteren, F.: Fully homomorphic simd operations. Design, Codes and Cryptography **71**(1), 57–81 (4 2014)
41. Stoer, J., Bulirsch, R.: Introduction to Numerical Analysis, vol. 12. Springer, 3 edn. (2002)
42. Zama: TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data (2022), https://github.com/zama-ai/tfhe-rs
43. Zhao, D.: Hermes: High-performance homomorphically encrypted vector databases (2025)

A Some CKKS algorithms and additional material

We present in this section some CKKS related operations including an high-level description of the (discrete) bootstrapping, reductions modulo 2 and the Kogge Stone adder (KSA) and the divide-et-impera multiplication pseudo codes.

A.1 The bootstrapping operation

Since after each multiplication, the scale of the resulting ciphertext is increased to Δ^2, a so-called rescaling operation is required. This is simply implemented as a product by $1/\Delta$, and by setting all the moduli in the chain as $q_i \approx \Delta$, we have that each multiplication *consumes* one moduli q_i in the chain $Q = \prod_i^L q_i$.

After $L - 1$ multiplications, the modulus becomes so small that no more rescaling operations can be executed, and in turn, multiplications. The bootstrapping operation in CKKS [14] raises the modulus back to larger values, and its core operation is the evaluation of (an approximation of) a modular reduction, since after the modulus raise some $q_1 \cdot k$ factors are added to the message, with k being some integer which magnitude depends on the secret key.

Recent advances in bootstrapping [3,2] showed how it is possible to include in the evaluation of the bootstrapping the evaluation of some other function. In

particular, we are interested in the *discrete bootstrapping* [3], whose goal is to both increase the modulus and reduce the amount of noise in discrete-CKKS ciphertexts. The main idea is as follows: instead of using a $\sin(x)$-like function to emulate the modular reduction, use a $\cos(x)$-like function assuming that all inputs are in the shape $\{0, 1\} + \varepsilon$ with $\varepsilon > 0$ relatively small. Now, given some proper scaling and shift of the $\cos(x)$ function, we end up with a bootstrapping function that, at the same time, removes the $q_i \cdot k$ terms (since $\cos(x)$ is periodic) but also reduces the amount of noise ε [3, Lemma 1].

A.2 Modulo 2 operation

In CKKS, there are several ways to reduce modulo 2 an addition. The fastest one is to evaluate $a \oplus b = (a - b)^2$, although this has the problem of potentially increasing the error contained in both a and b [19, Lemma 2]. On the other hand, we also propose another polynomial that, at the cost of an additional multiplicative level, allows one to reduce modulo 2 and, at the same time, to apply a cleaning operation to the error. The polynomial is

$$h(x) := x^2(x-2)^2, \quad a \oplus b := h(a+b). \tag{4}$$

We present its plot in Figure 6. Also observe the error shrinks quadratically, as

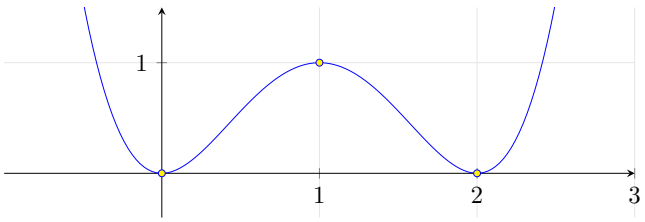


Fig. 6: The $x^2 \cdot (x-2)^2$ polynomial in the $(-1, 3)$ interval

shown in the following Lemma 6.

Lemma 6. *Let $h(x) = x^2(x-2)^2$. For each $x \in \{0, 1, 2\}$, there exist a constant $k > 0$ such that for all real e with $|e| < 0.1$, we have*

$$h(x + e) = h(x) + e',$$

where the new error e' satisfies

$$|e'| \leq ke^2.$$

Proof. Consider $x = 0$. We have

$$h(e) = e^2(e-2)^2$$

for $|e| < 0.1$, we can bound $e - 2$ as

$$|e - 2| < 2.1$$

leading to

$$|h(e)| < 4.41e^2$$

The same estimate holds for $x = 2$ by symmetry as $h(x) = h(2 - x)$. Lastly, for $x = 1$, we compute

$$h(1 + e) = (1 - e^2)^2 = 1 + e^4 - 2e^2.$$

Thus the new error is

$$e' = e^4 - 2e^2.$$

Using the triangle inequality,

$$|e'| \leq |e|^4 + 2|e|^2.$$

Since $|e| < 0.1$, we have $|e|^2 < 0.01$ and $|e|^4 < 0.0001$, so

$$|e'| < 0.0001 + 0.02 < 3e^2.$$

□

A.3 Kogge Stone adder for CKKS

As previously mentioned, additions are performing using the Kogge Stone adder (KSA) [30], described in Algorithm 1. In Algorithm 3 we give another version of the same algorithm, but specifically written using CKKS primitives.

Algorithm 3 Kogge Stone adder based on SIMD operations

Require: Two n-bit numbers $\boldsymbol{a} = (a_0, \ldots, a_{n-1})$, $\boldsymbol{b} = (b_0, \ldots, b_{n-1})$
Ensure: Sum $\boldsymbol{s} = (s_0 \ldots s_{n-1})$ and carry-out s_n
1: *Define the propagate ($\boldsymbol{p}$) and generate ($\boldsymbol{v}$) vectors:*
2: $\boldsymbol{p} := (\boldsymbol{a} - \boldsymbol{b})^2, \quad \boldsymbol{g} := \boldsymbol{a} \cdot \boldsymbol{b}$
3: *Compute prefix carries using Kogge-Stone parallel prefix:*
4: **for** $d = 0$ **to** $\lceil \log_2 n \rceil - 1$ **do**
5: $\quad \boldsymbol{g} := \boldsymbol{g} + (\boldsymbol{p} \cdot \mathrm{Rot}_{-2^d}(\boldsymbol{g}))$
6: $\quad \boldsymbol{p} := \boldsymbol{p} \cdot \mathrm{Rot}_{-2^d}(\boldsymbol{p})$
7: **end for**
8: *Post processing:*
9: $\boldsymbol{c} := \mathrm{Rot}_{-1}(\boldsymbol{g})$
10: $\boldsymbol{s} := ((\boldsymbol{a} - \boldsymbol{b})^2 - \boldsymbol{c})^2$
11: **return**

Algorithm 4 Comparison

Require: Two n-bit numbers $\boldsymbol{a} = (a_0, \ldots, a_{n-1})$, $\boldsymbol{b} = (b_0, \ldots, b_{n-1})$
Ensure: Comparison as $a \leq b$ in c_n
1: *Invert the bits of* $\boldsymbol{b}$*:*
2: $\hat{\boldsymbol{b}}_i := 1 - \boldsymbol{b}_i$
3: $\boldsymbol{c} := \mathrm{KSA}(\boldsymbol{a}, \hat{\boldsymbol{b}})$
4: **return** c_n

A.4 Comparisons for CKKS

As mentioned in Section 3.2, we can evaluate a comparison with the same complexity as a KSA addition. We give in Algorithm 4 a formal definition of the procedure, which returns either 1 or 0 in the $(n+1)$-th bit of the result.

A.5 Divide-et-impera multiplication for CKKS

We provide, in Algorithm 5, a version of the divide-et-impera mulitplication that makes only a single recursive call at each step, taking advantage of SIMD computations. We kept the algorithm at a high-level, although we refer to the notebooks available in the open-source repository for some plain implementations of it.

Notice that, in practice, the multiplications required in the first $\log(n)$ recursive calls can be done in a single processing when $\log(n) = 8$ to save some levels. This is the strategy that we followed in our code, refer to the two notebooks in the repository for the two different strategies.

B On the role of Carry-save adder (CSA)

In our divide-et-impera multiplication, given the four multiplications:

- $2^n \cdot \boldsymbol{a}_h \cdot \boldsymbol{b}_h$,
- $2^{n/2} \cdot \boldsymbol{a}_h \cdot \boldsymbol{b}_\ell$,
- $2^{n/2} \cdot \boldsymbol{a}_\ell \cdot \boldsymbol{b}_h$,
- $\boldsymbol{a}_\ell \cdot \boldsymbol{b}_\ell$,

we are required to add them – with carry – to obtain the final result. Notice that the multiplications by powers of two can be implemented using shifts (Section 3.4), although we would need to evaluate at least two additions with carry to obtain the final addition result. Instead, we use the carry-save adder (CSA), that, given $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}$, returns $\boldsymbol{c}', \boldsymbol{s}$ such that $\boldsymbol{c}' + \boldsymbol{s} = \boldsymbol{a} + \boldsymbol{b} + \boldsymbol{c}$. Since our operands are four, we can evaluate two composed CSA (which are less expensive than an addition with carry) to produce two operands that, once added with KSA, give as output the result of the final addition. We start by providing a definition of the CSA in Algorithm 6.

Algorithm 5 Divide-et-impera multiplier based on SIMD operations (high-level)

Require: Two n-bit numbers $\boldsymbol{a} = (a_0, \ldots, a_{n-1})$, $\boldsymbol{b} = (b_0, \ldots, b_{n-1})$
Ensure: Multiplication $\boldsymbol{m} = (m_0 \ldots m_{n^2-1})$
Procedure multSIMD($\boldsymbol{a}, \boldsymbol{b}, n$)
if $n = 4$ **then**
    All the base cases are computed in one operation in parallel
    return $\mathbb{R}$-to-$\mathbb{Z}(\mathbb{Z}$-to-$\mathbb{R}(\boldsymbol{a}) \cdot \mathbb{Z}$-to-$\mathbb{R}(\boldsymbol{b}))$
else
    $\boldsymbol{m}_\ell :=$ mask for low bits, contains ones from 0 to $n/2$ (excl.)
    $\boldsymbol{m}_h :=$ mask for high bits, contains ones from $n/2$ to n (excl.)
    Replicating $\boldsymbol{a}_h$:
    $\boldsymbol{a}' := \boldsymbol{a} \cdot \boldsymbol{m}_h + \mathrm{Rot}_{-n/2}(\boldsymbol{a} \cdot \boldsymbol{m}_h)$
    And adding two times $\boldsymbol{a}_\ell$:
    $\boldsymbol{a}' := \boldsymbol{a}' + \mathrm{Rot}_{-n^2-n}(\boldsymbol{a} \cdot \boldsymbol{m}_\ell) + \mathrm{Rot}_{-n^2}(\boldsymbol{a} \cdot \boldsymbol{m}_\ell)$
    Similarly for $\boldsymbol{b}$:
    $\boldsymbol{b}' := \mathrm{Rot}_{n/2}(\boldsymbol{b} \cdot \boldsymbol{m}_h) + \mathrm{Rot}_{-n}(\boldsymbol{b} \cdot \boldsymbol{m}_\ell) + \mathrm{Rot}_{-n/2}(\boldsymbol{b} \cdot \boldsymbol{m}_h) + \mathrm{Rot}_{-n^2-n}(\boldsymbol{b} \cdot \boldsymbol{m}_\ell)$
    Now we have $\boldsymbol{a}' = (\boldsymbol{a}_h, 0, \boldsymbol{a}_h, 0, \boldsymbol{a}_\ell, 0, \boldsymbol{a}_\ell, 0)$ and $\boldsymbol{b}' = (\boldsymbol{b}_h, 0, \boldsymbol{b}_\ell, 0, \boldsymbol{b}_h, 0, \boldsymbol{b}_\ell, 0)$. The size of zeros is $n/2$ and each will be filled in the next recursive calls
    $\boldsymbol{m} :=$ multSIMD($\boldsymbol{a}', \boldsymbol{b}', n/2$)
    The partial results are shifted and added with CSA. The mask *function masks the values of interest, depending on the term*
    $\boldsymbol{t}_1 :=$ mask($\boldsymbol{m}$)
    $\boldsymbol{t}_2 := \mathrm{Rot}_{-n/2}(\mathrm{mask}(\boldsymbol{m}))$
    $\boldsymbol{t}_3 := \mathrm{Rot}_{-n/2}(\mathrm{mask}(\boldsymbol{m}))$
    $\boldsymbol{t}_4 := \mathrm{Rot}_{-n}(\mathrm{mask}(\boldsymbol{m}))$
    return CSA($\boldsymbol{t}_1, \boldsymbol{t}_2, \boldsymbol{t}_3, \boldsymbol{t}_4$) ▷ Refer to Algorithm 7.
end if

Algorithm 6 Carry-save adder (CSA) based on SIMD operations

Require: Three n-bit numbers $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}$
Ensure: Sum $\boldsymbol{s}$ and carry $\boldsymbol{c}$ such that $\boldsymbol{s} + \boldsymbol{c} = \boldsymbol{a} + \boldsymbol{b} + \boldsymbol{c}$
Compute the first XOR:
$\boldsymbol{s} := (\boldsymbol{a} - \boldsymbol{b})^2$
Compute the second XOR:
$\boldsymbol{s} := (\boldsymbol{s} - \boldsymbol{c})^2$
Compute the majority bits:
$\boldsymbol{t} := \boldsymbol{a} + \boldsymbol{b} + \boldsymbol{c}$
$\boldsymbol{c} := \frac{1}{3}\boldsymbol{t}^3 + \frac{3}{2}\boldsymbol{t}^2 - \frac{7}{6}\boldsymbol{t}$
We rotate by one the carry to move it in the correct position:
return $\boldsymbol{s}, \mathrm{Rot}_{-1}(\boldsymbol{c})$

Lemma 7. *The multiplicative depth of the CSA described in Algorithm 6 is equal to 2.*

Proof. Lines 2-4, and 7 each require 2 multiplicative levels, but line 7 can be computed independently. Hence, the overall depth is 2. □

Notice that the last polynomial $p(x) := \frac{1}{3}x^3 + \frac{3}{2}x^2 - \frac{7}{6}x$ returns 0 in positions in which the sum is either 0 or 1, and returns 1 otherwise (sum equal to 2 or 3). We provide a plot of $p(x)$ for convenience in Figure 7.

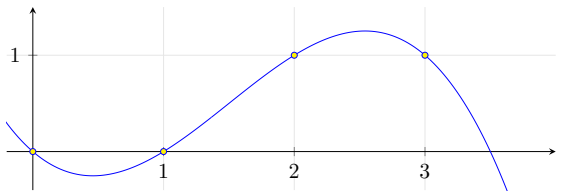


Fig. 7: The $\frac{1}{3}x^3 + \frac{3}{2}x^2 - \frac{7}{6}x$ polynomial in the $[0, 4)$ interval

Next, we can use this as a building block for our four operands adder. We present the final procedure in Algorithm 7. To conclude, observe that the muli-

Algorithm 7 Reducing $a+b+c+d$ to a single KSA, based on SIMD operations

Require: Four n-bit numbers $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}, \boldsymbol{d} \in \{0,1\}^n$
Ensure: Sum $\boldsymbol{s} = (s_0 \dots s_{n-1})$ and carry-out s_n
1: *Compute the first CSA:*
2: $\boldsymbol{s}_1, \boldsymbol{c}_1 := \mathrm{CSA}(\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c})$
3: *Compute the second CSA:*
4: $\boldsymbol{s}_2, \boldsymbol{c}_2 := \mathrm{CSA}(\boldsymbol{s}_1, \boldsymbol{c}_1, \boldsymbol{d})$
5: *Return the actual addition with carry:*
6: **return** $\mathrm{KSA}(\boldsymbol{s}_2, \boldsymbol{c}_2)$

plicative cost of two KSA is equal to

$$2(2 + \log(d)), \tag{5}$$

by Lemma 4, while the cost of two CSA (Lemma 7), followed by a KSA, is

$$4 + 2 + \log(d). \tag{6}$$

In general, we have that performing two KSA is more convenient when Eq. (5) is smaller than Eq. (6), that is when

$$2(2 + \log(d)) < 4 + 2 + \log(d).$$

This holds as long as $0 < d < 4$, and for our cases we are even outside of the base case of the multiplier ($d = 4$). Hence, it is always convenient to use two CSA and one KSA.

C Additional experiments with dedicated parameter sets

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

C.1 Special primes

Our framework well-suits powers of two, although many cryptographic applications typically rely on particular prime moduli, such as $p = 2^{255} - 19$ or $p = 2^{64} - 2^{32} + 1$, and so on. We observe that, although not natively supported, our framework can be adapted to work on those moduli. The key is that typically these moduli are chosen because of their "nice" structure. For instance, one could work in $p = 2^{255} - 19$ by working with 256-bits and, for example, applying a subsequent operation where the first 255 bits are kept and the remaining bit is multiplied by 19 and added. This requires a simple addition with a carry – i.e., at most one additional bootstrapping. In practice, we can benefit from the choices originally made when choosing those particular moduli.

Notice that, e.g., in [10], using $p = 2^{255} - 19$ is more than three times more expensive than working in 256-bits [10, Table 6], in our case this can cost at most one more addition (so approximately 4.5 seconds more). We keep the door open to experiments with other moduli for future work.

D Proof of lemmas

D.1 Lemma 1

Proof. Let $\boldsymbol{x} = (x_0, \ldots, x_{k-1}) \in \{0,1\}^k$ and $\boldsymbol{w} = (2^0, \ldots, 2^{k-1})$. Compute the SIMD product

$$\boldsymbol{y} = (x_0 2^0, \ldots, x_{k-1} 2^{k-1}).$$

A rotate-and-sum procedure requiring $\log(k)$ rotations and additions accumulates all entries of $\boldsymbol{y}$ into the first slot, yielding

$$\sum_{i=0}^{k-1} x_i 2^i = \mathrm{dec}(\boldsymbol{x}).$$

□

D.2 Lemma 2

Proof. Write $x \in \mathbb{Z}^+$ in binary:

$$x = \sum_{i=0}^{n-1} 2^i x_i, \qquad x_i \in \{0, 1\}.$$

Divide by 2^k:

$$\frac{x}{2^k} = \sum_{i=0}^{n-1} 2^{i-k} x_i = \sum_{i=0}^{k-1} 2^{i-k} x_i + \sum_{i=k}^{n-1} 2^{i-k} x_i.$$

The first sum contains negative powers of 2 and is strictly less than 1. The second sum is an integer:

$$\left\lfloor \frac{x}{2^k} \right\rfloor = \sum_{i=k}^{n-1} 2^{i-k} x_i.$$

Now take modulo 2, the coefficient of 2^0 in the integer sum is exactly x_k. Hence,

$$x_k = \left\lfloor \frac{x}{2^k} \right\rfloor \bmod 2.$$

□

D.3 Lemma 3

Proof. A possible algorithm is as follows. Let $\boldsymbol{x}$ contain k repetitions of $x \in \mathbb{Z}_{2^k}$:

$$\boldsymbol{x} := (x, x, \ldots, x) \in \mathbb{R}^k$$

Let $f_i(x)$ be a binary i-periodic function (Definition 3). Define $\boldsymbol{x}_{\text{bin}}$ as

$$(\boldsymbol{x}_{\text{bin}})_i := f_i(x_i)$$

The result follows by considering Corollary 1. □

D.4 Corollary 2

Proof. By Theorem 1, $p(x)$ interpolates at $x_i = 2i\pi/N$. Set $N = 2P$ and $f(x) = \lfloor (x \cdot N)/(2^{k+1} \cdot \pi) \rfloor \bmod 2$. Now $p(x)$ is such that

$$\begin{aligned}
p(x_0) &= f(x_0) = 0 \\
p(x_1) &= f(x_1) = \left\lfloor \frac{2\pi \cdot N}{N \cdot 2^{k+1} \cdot \pi} \right\rfloor \equiv \left\lfloor \frac{1}{2^k} \right\rfloor \bmod 2 \\
p(x_2) &= f(x_2) = \left\lfloor \frac{2 \cdot 2\pi \cdot N}{N \cdot 2^{k+1} \cdot \pi} \right\rfloor \equiv \left\lfloor \frac{2}{2^k} \right\rfloor \bmod 2 \\
&\vdots
\end{aligned}$$

In general, $p(x_i) = \lfloor i/2^k \rfloor \bmod 2$. □

D.5 Lemma 4

Proof. Consider Algorithm 3. line 2 requires a single multiplicative depth. Within the loop, each iteration involves two multiplications whose inputs are independent. Formally, each multiplication produces a multivariate polynomial of degree 2, and the two results can be viewed as components of a vector of multivariate polynomials evaluated at the same level; thus, each iteration contributes only one additional multiplicative depth.

The loop runs for $\log(n)$ iterations, resulting in a total depth contribution of $\log(n)$. Finally, line 10 applies an XOR operation to the carry vector $\boldsymbol{s}$, which is already at level $1 + \log(n)$; this adds one more level. Therefore, the total multiplicative depth of the algorithm is

$$1 + \log(n) + 1 = 2 + \log(n).$$

□

D.6 Lemma 5

Proof. Consider Algorithm 5 and let $\boldsymbol{a}, \boldsymbol{b} \in \{0,1\}^n$ be the two operands. At each recursive call we expand the size of the operands from n to $4n$ as:

$$\boldsymbol{a}' := (\boldsymbol{a}_h, \mathbf{0}, \boldsymbol{a}_h, \mathbf{0}, \boldsymbol{a}_\ell, \mathbf{0}, \boldsymbol{a}_\ell, \mathbf{0}) \in \{0,1\}^{4n},$$
$$\boldsymbol{b}' := (\boldsymbol{b}_h, \mathbf{0}, \boldsymbol{b}_\ell, \mathbf{0}, \boldsymbol{b}_h, \mathbf{0}, \boldsymbol{b}_\ell, \mathbf{0}) \in \{0,1\}^{4n},$$

where $\boldsymbol{a} = \boldsymbol{a}_\ell \,||\, \boldsymbol{a}_h$ and $\boldsymbol{b} = \boldsymbol{b}_\ell \,||\, \boldsymbol{b}_h$, with subvectors of size $n/2$.

Since the recursion splits into four independent calls (merged in a single call with SIMD, but the required space does not change) of size $n/2$, the total number of required slots $S(n)$ satisfies the recurrence

$$S(n) = 4 \cdot S(n/2) \quad \text{for } n > 4, \quad \text{with } S(4) = 4.$$

Solving this recurrence, noting that $n = 2^k$, we have

$$S(n) = 4 \cdot 4 \cdot 4 \cdots = 4^{\log_2(n/4)} \cdot S(4) = 4^{\log_2(n/4)} \cdot 4 = \frac{n^2}{2}.$$

Hence, an n-bit multiplication requires exactly $n^2/2$ slots to be executed in parallel. □